A Better Approximation for Interleaved Dyck Reachability

GIOVANNA KOBUS CONRADO
Hong Kong University of Science and Technology

ANDREAS PAVLOGIANNIS
Aarhus University

SOAP 2024

Interleaved Dyck Reachability

Problem

Can the value of variable **a** flow into variable **c**?

This type of problem is found in:

- Data-flow analysis
- Taint analysis
- Data-dependence analysis
- Alias analysis
- ...

One set of benchmarks

- We will test the algorithms on a taint analysis benchmark on Android
- Detect data flows from sensitive data (phone location) to untrusted sinks (logs to apps)

Wei Huang, Yao Dong, Ana Milanova, and Julian Dolby. 2015. **Scalable and Precise Taint Analysis for Android**. ISSTA 2015.

How do we solve this?

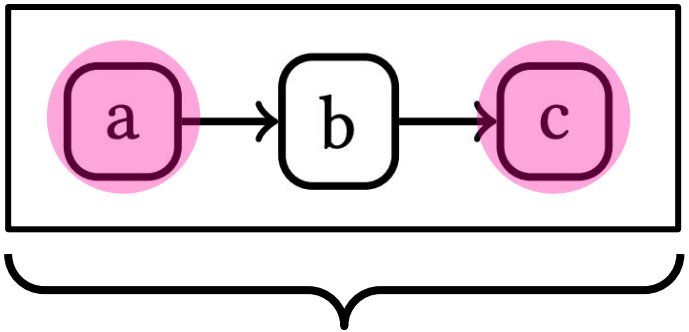
Can the value of variable **a** flow into variable **c**?

$$\begin{aligned} &\text{int} \quad a \leftarrow 2 \\ &\text{int} \quad b \leftarrow a \\ &\text{int} \quad c \leftarrow b \end{aligned}$$

How do we solve this?

Can the value of variable **a** flow into variable **c**?

$$\begin{aligned} \text{int} \quad & a \leftarrow 2 \\ \text{int} \quad & b \leftarrow a \\ \text{int} \quad & c \leftarrow b \end{aligned}$$

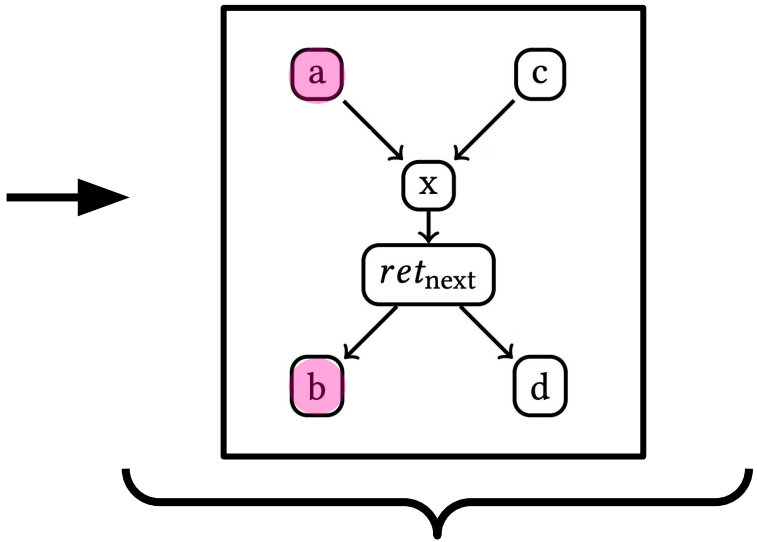


What if there are function calls?

```
fun next(x)
    return x+1

fun next2()
    int a ← 2
    int b ← next(a)

fun next3()
    int c ← 3
    int d ← next(c)
```

What if there are function calls?

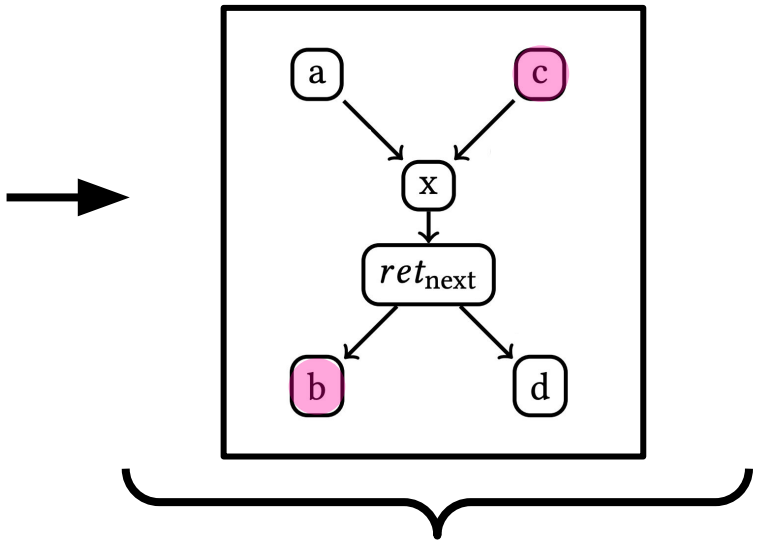
```
fun next(x)
    return x+1

fun next2()
    int a ← 2
    int b ← next(a)

fun next3()
    int c ← 3
    int d ← next(c)
```



a reaches **b**

What if there are function calls?

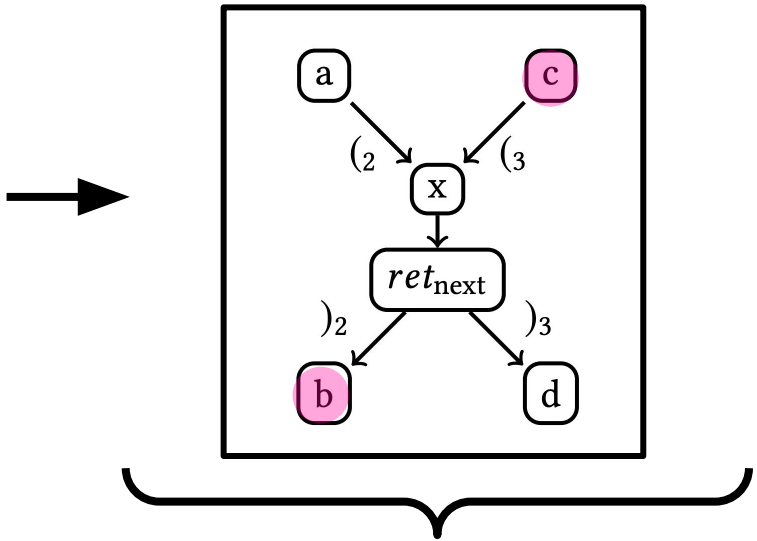
```
fun next(x)
    return x+1

fun next2()
    int a ← 2
    int b ← next(a)

fun next3()
    int c ← 3
    int d ← next(c)
```



a reaches **b**... but so does **c**

What if there are function calls?

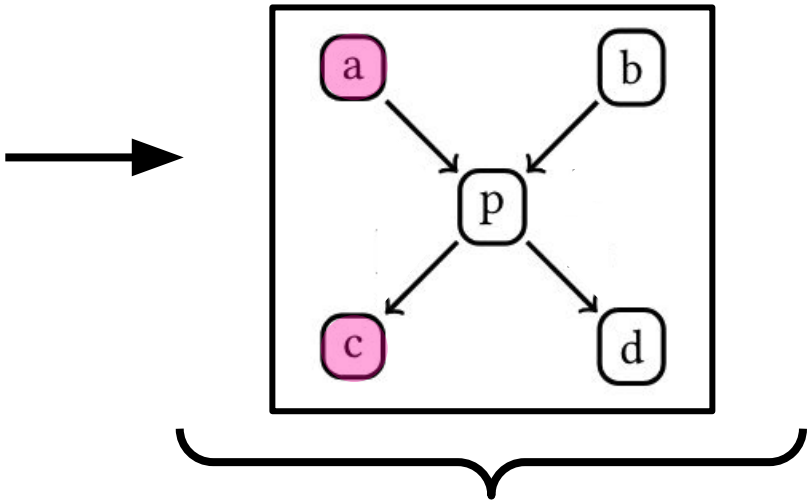
```
fun next(x)
    return x+1

fun next2()
    int a ← 2
    int b ← next(a)

fun next3()
    int c ← 3
    int d ← next(c)
```



the path must be well-nested

What if there are composite data types?

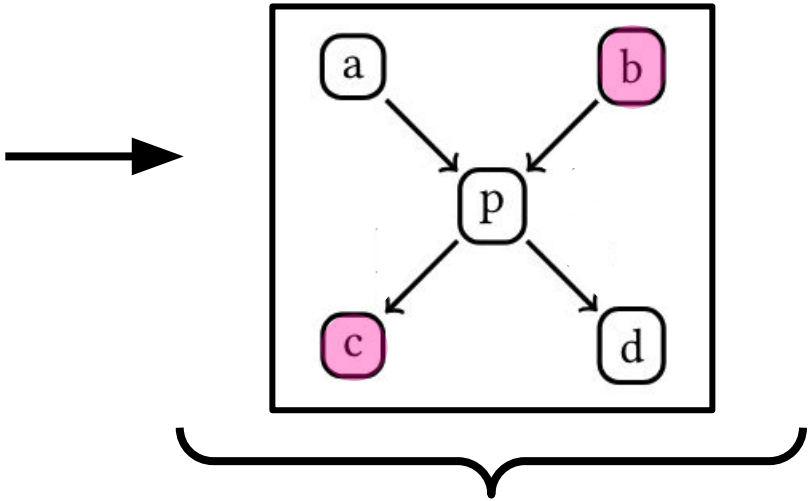
```
int  a ← 2
int  b ← 3
point  p
p.x ← a
p.y ← b
int  c ← p.x
int  d ← p.y
```



a reaches **c**

What if there are composite data types?

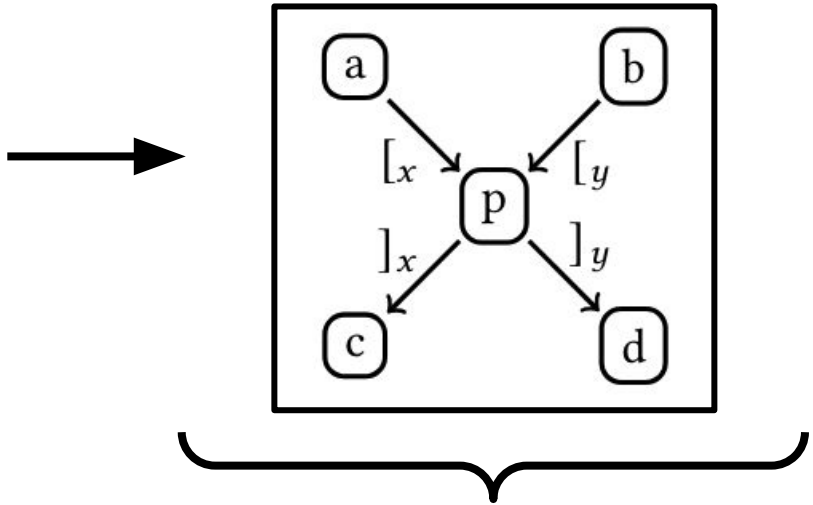
```
int  a ← 2
int  b ← 3
point  p
p.x ← a
p.y ← b
int  c ← p.x
int  d ← p.y
```



a reaches **c**... but so does **b**

What if there are composite data types?

```
int  a ← 2
int  b ← 3
point  p
p.x ← a
p.y ← b
int  c ← p.x
int  d ← p.y
```



the path must be well-nested

How do we check if a path is well-nested?

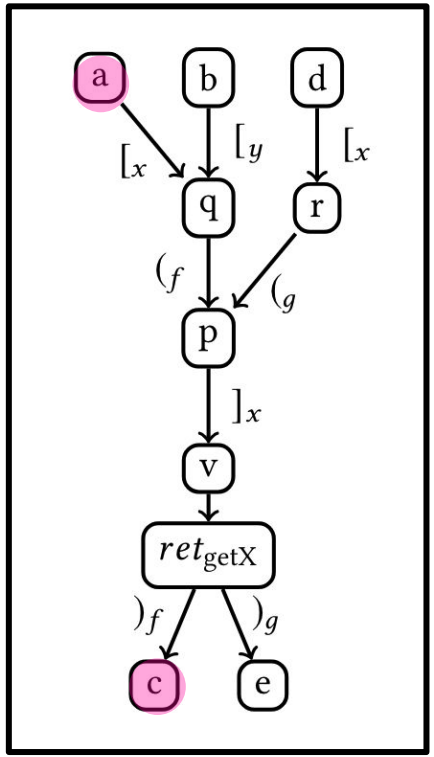
Well-nested parenthesis → Dyck language → Context-free language

$$S(\epsilon)$$
$$S((_i x)_i) \leftarrow S(x)$$
$$S(x_1 x_2) \leftarrow S(x_1), S(x_2)$$

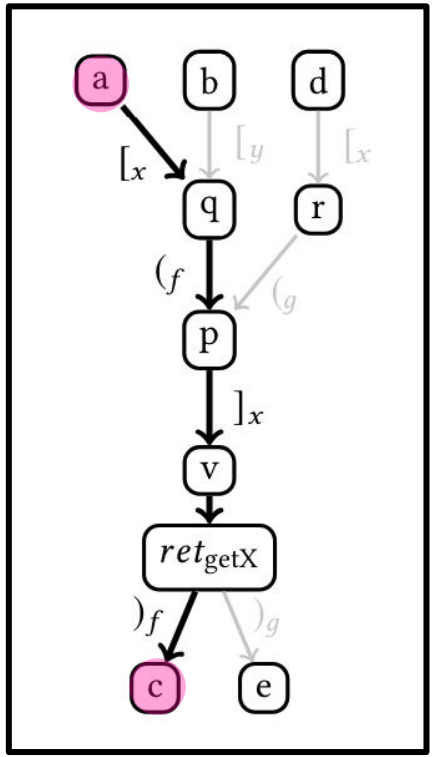
We can identify all CFL-reachable pairs in $O(n^3)$

How do we handle context and field sensitivity simultaneously?

```
fun getX(p)
    int v ← p.x
    return v

fun f()
    int a ← 2
    int b ← 3
    point q
    q.x ← a
    q.y ← b
    int c ← getX(q)

fun g()
    int d ← 1
    point r
    r.x ← d
    int e ← getX(r)
```



```
fun getX(p)
    int v ← p.x
    return v

fun f()
    int a ← 2
    int b ← 3
    point q
    q.x ← a
    q.y ← b
    int c ← getX(q)

fun g()
    int d ← 1
    point r
    r.x ← d
    int e ← getX(r)
```



$[_x\ (_f\ ]_x\ )_f$

$[_x\ \mathbf{(_f}\ ]_x\ \mathbf{)_f}$

$\mathbf{[_x}\ (_f\ \mathbf{]_x}\ )_f$

How do we check if a path is well-nested?

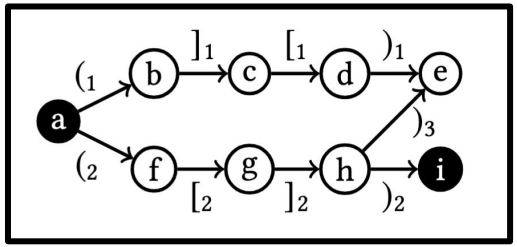
Interleaving of two Dyck languages → Interleaved Dyck Language

Interleaved Dyck Reachability? **Undecidable**

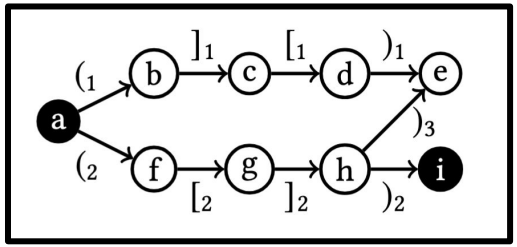
We seek to overapproximate the number of reachable pairs.

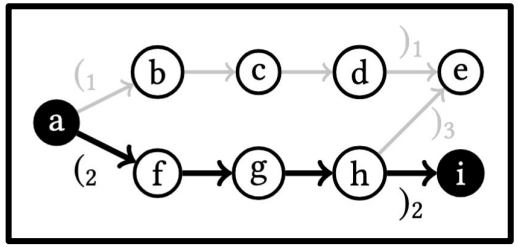
Current approaches

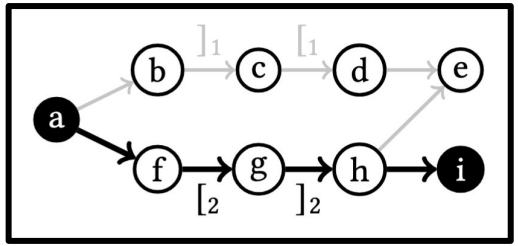
Intersection



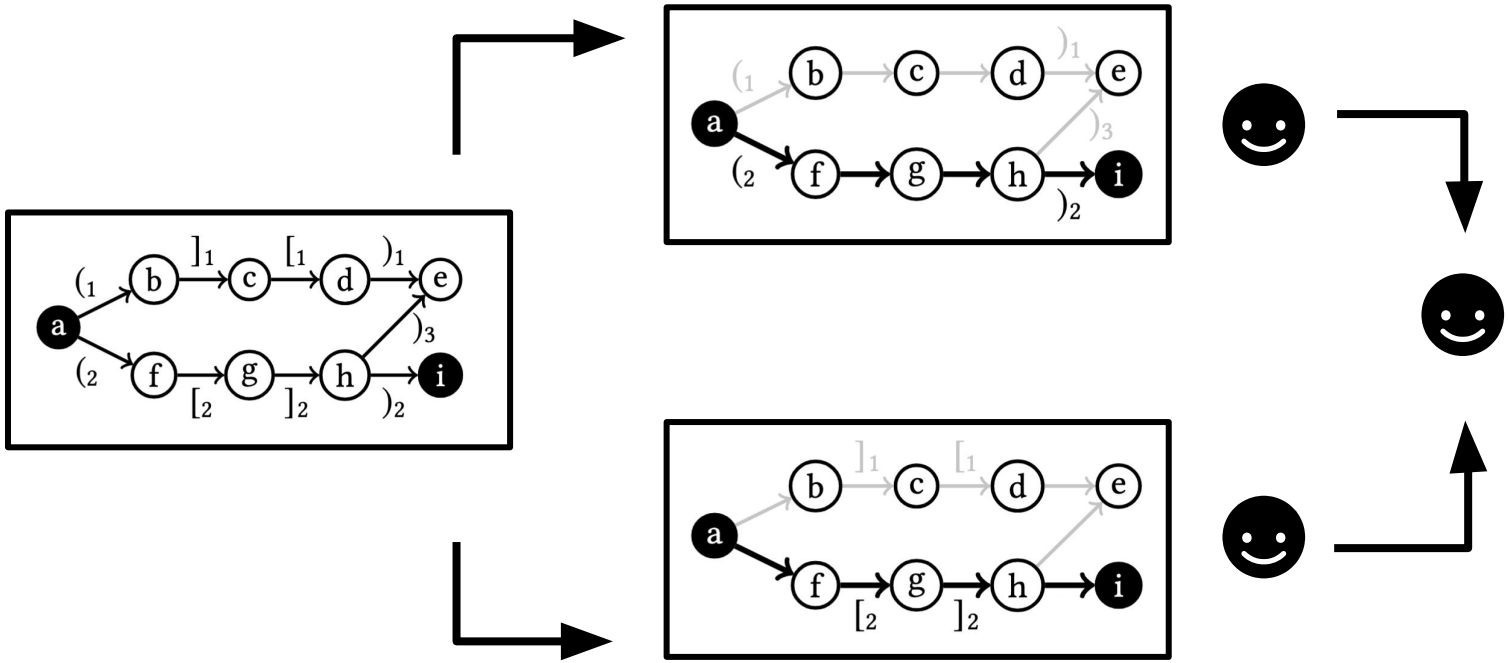
Intersection



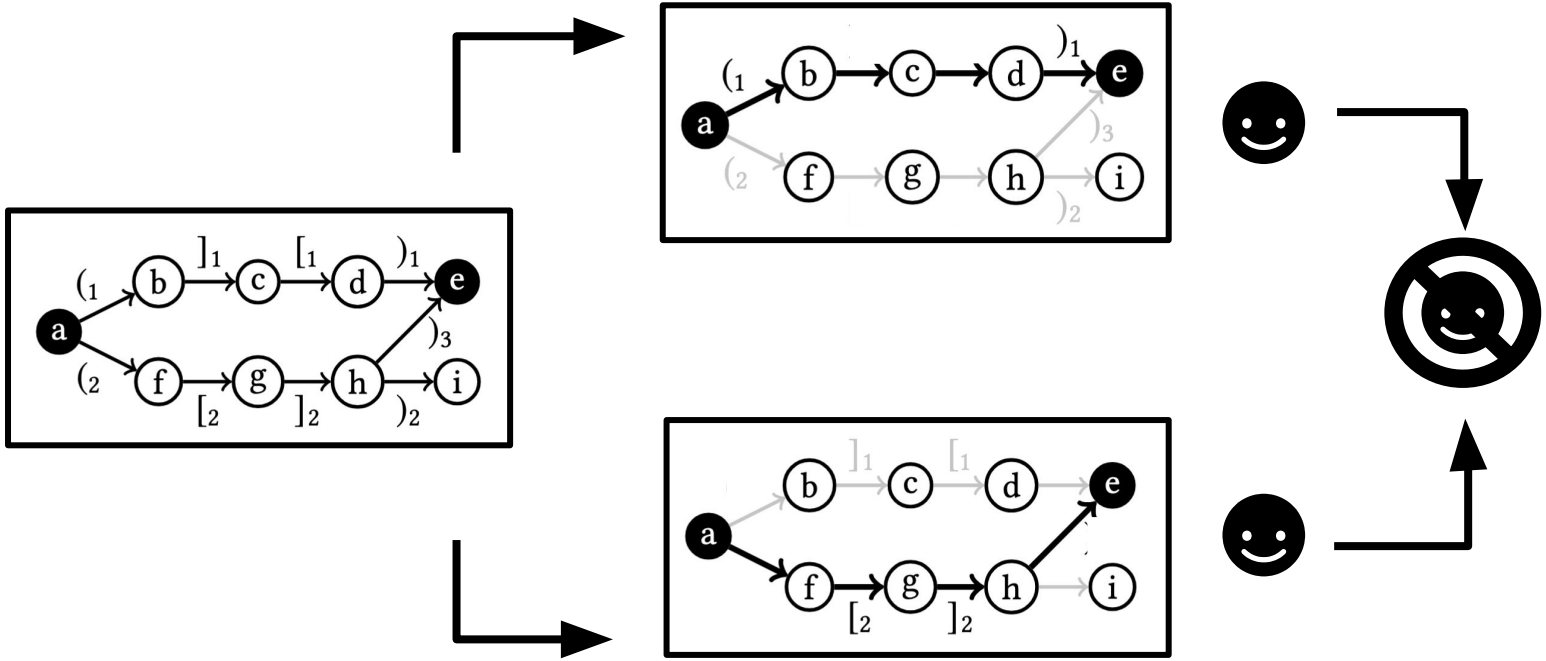


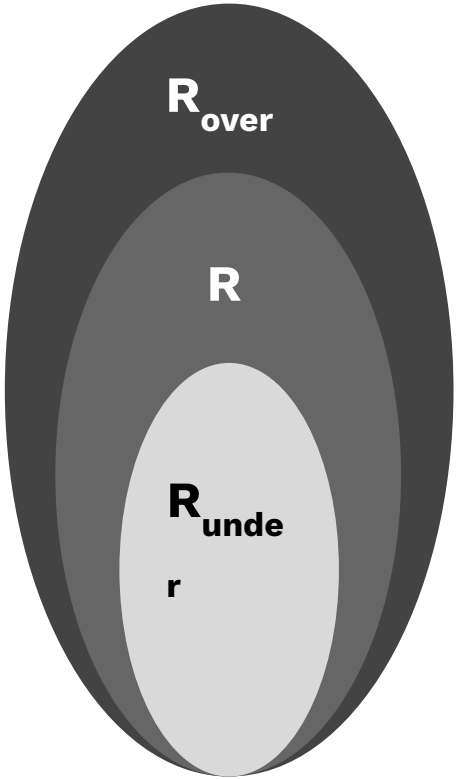


Intersection



Intersection



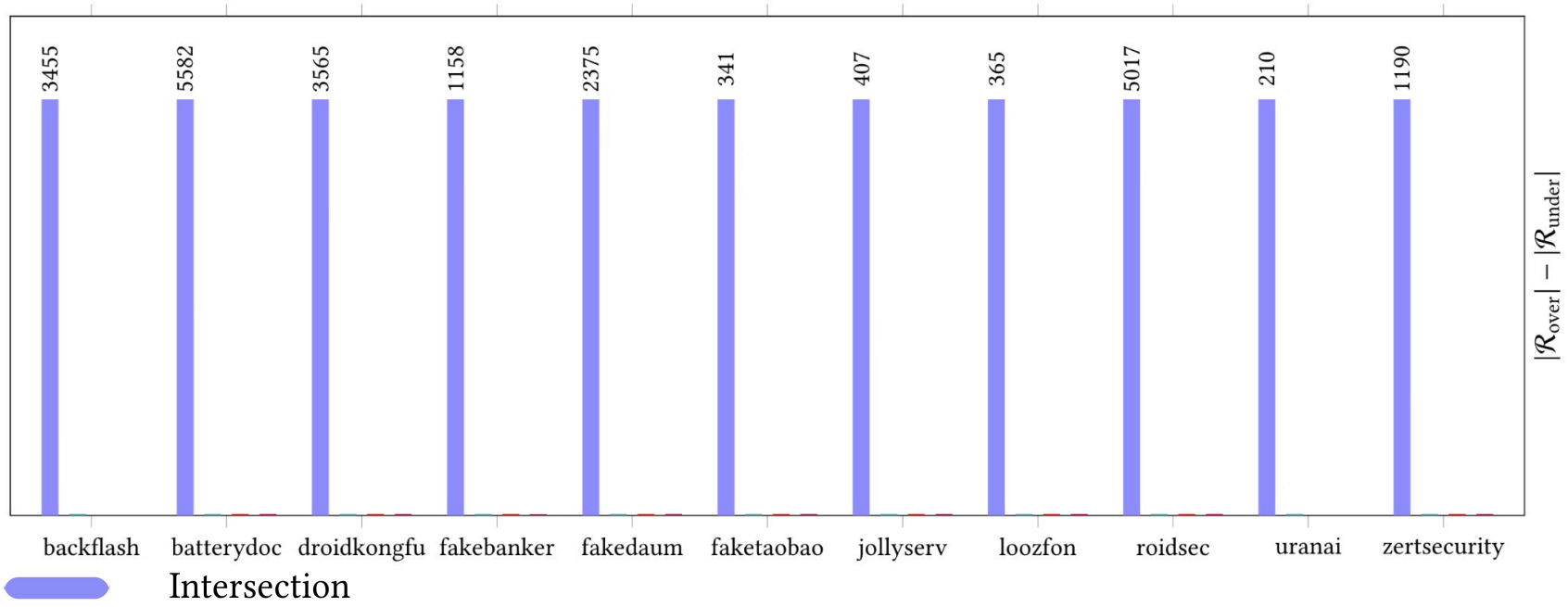


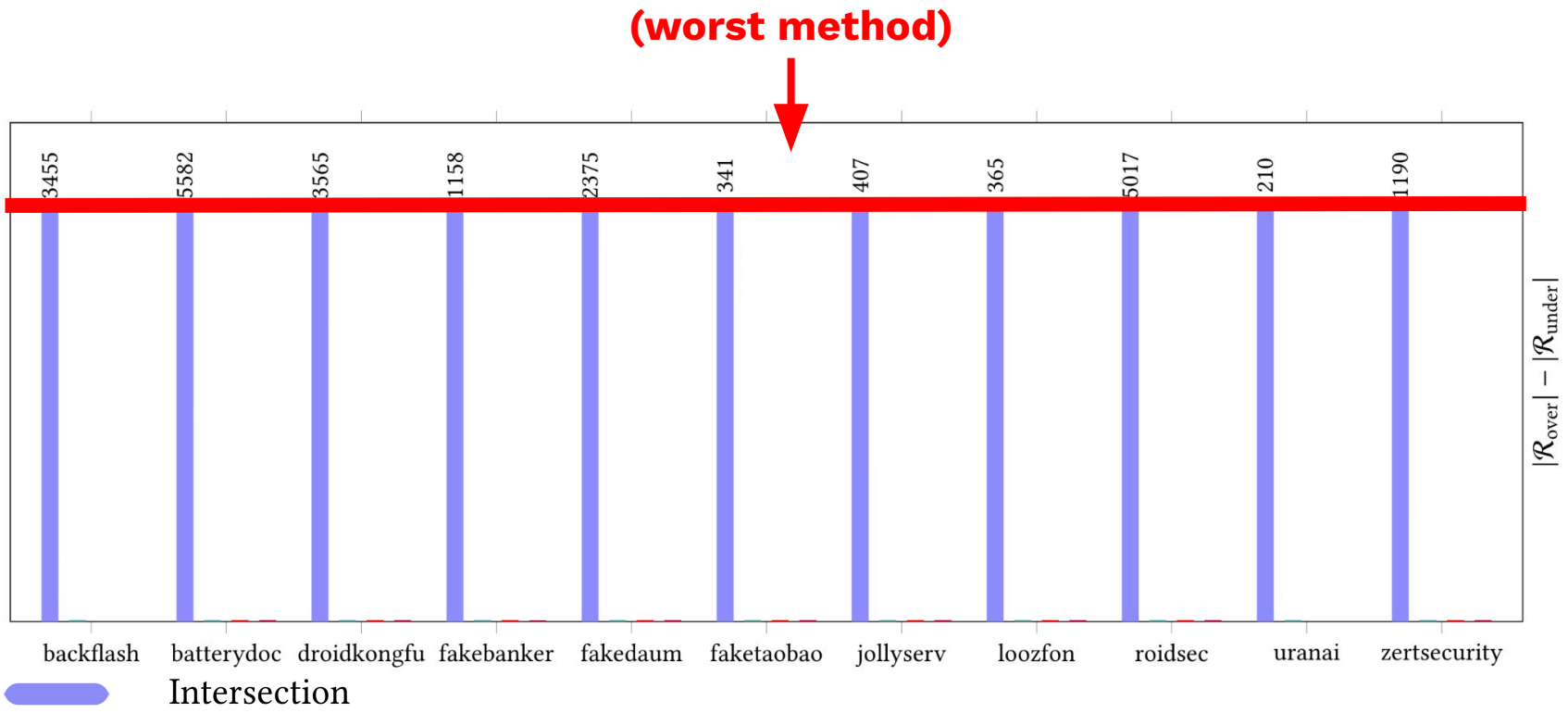
Each algorithm will present us with an **overapproximation** of reachable node pairs $\mathbf{R_{over}}$

We also calculated an **underapproximation** of reachable node pairs $\mathbf{R_{under}}$

We will compare the potential false positives $|R_{over}| - |R_{under}|$

Potential false positives per benchmark (normalized by worst method)

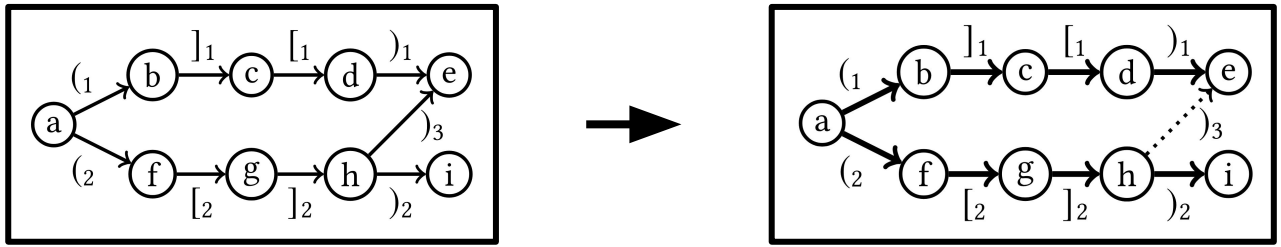



(worst method)
3455
5582
3565
1158
2375
341
407
365
5017
210
1190
backflash
batterydoc
droidkongfu
fakebanker
fakedaum
faketaobao
jollyserv
loozfon
roidsec
uranai
zertsecurity
Intersection
$|\mathcal{R}_{over}| - |\mathcal{R}_{under}|$

Mutual refinement*

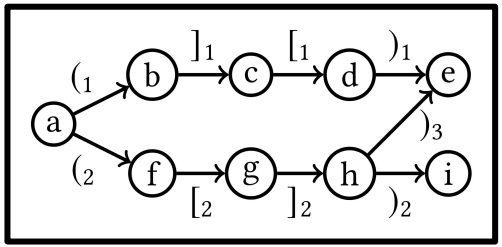
Works by simplifying the graph

Iteratively remove edges that cannot be in any answer

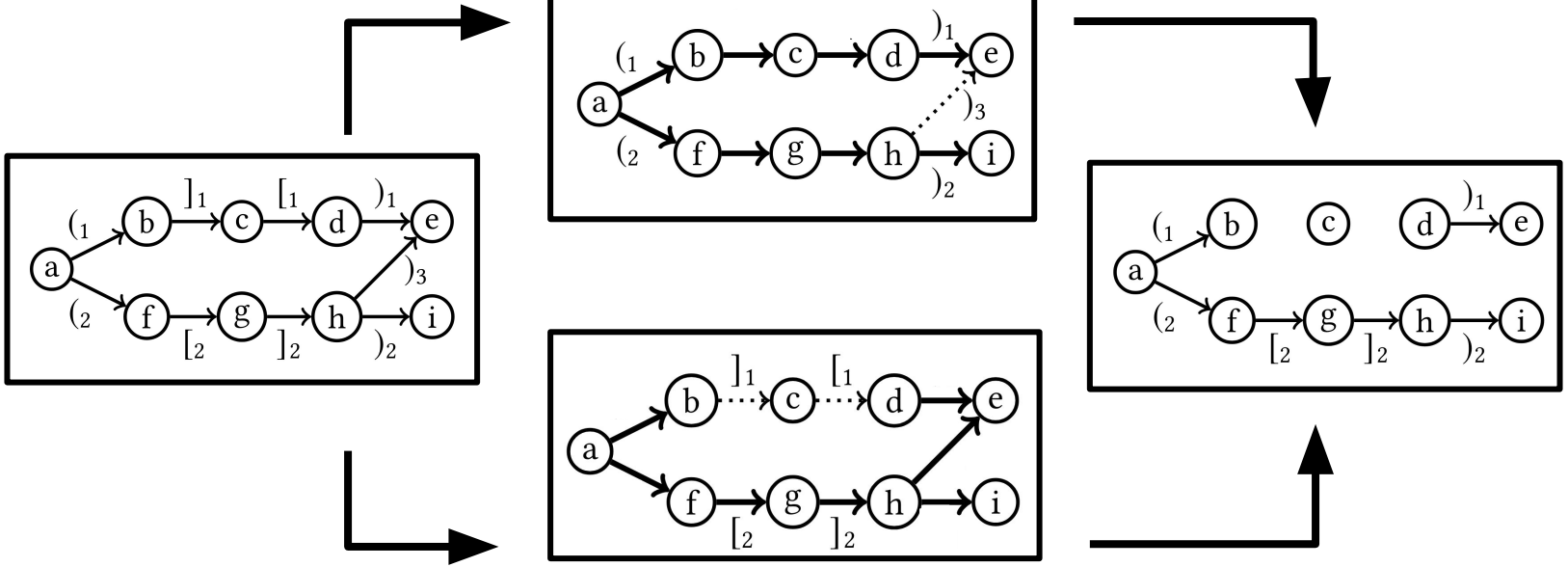


*Shuo Ding and Qirun Zhang. 2023. **Mutual Refinements of Context-Free Language Reachability**. SAS 2023

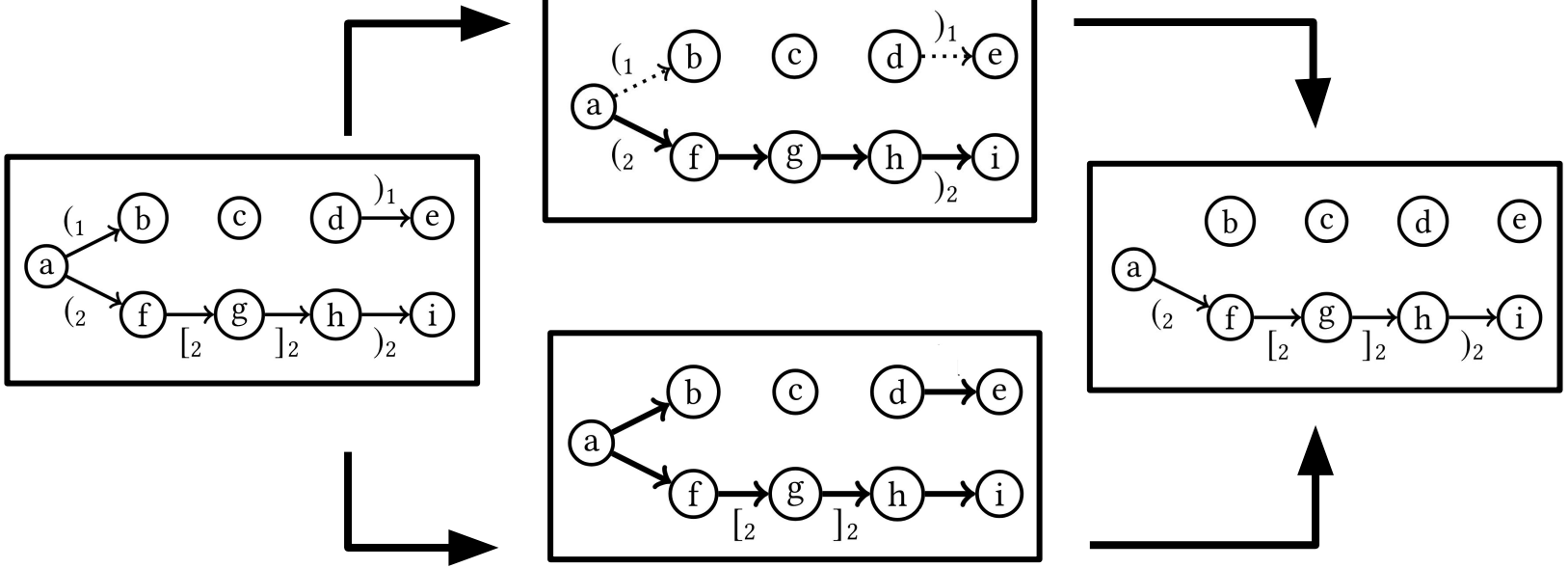
Mutual refinement



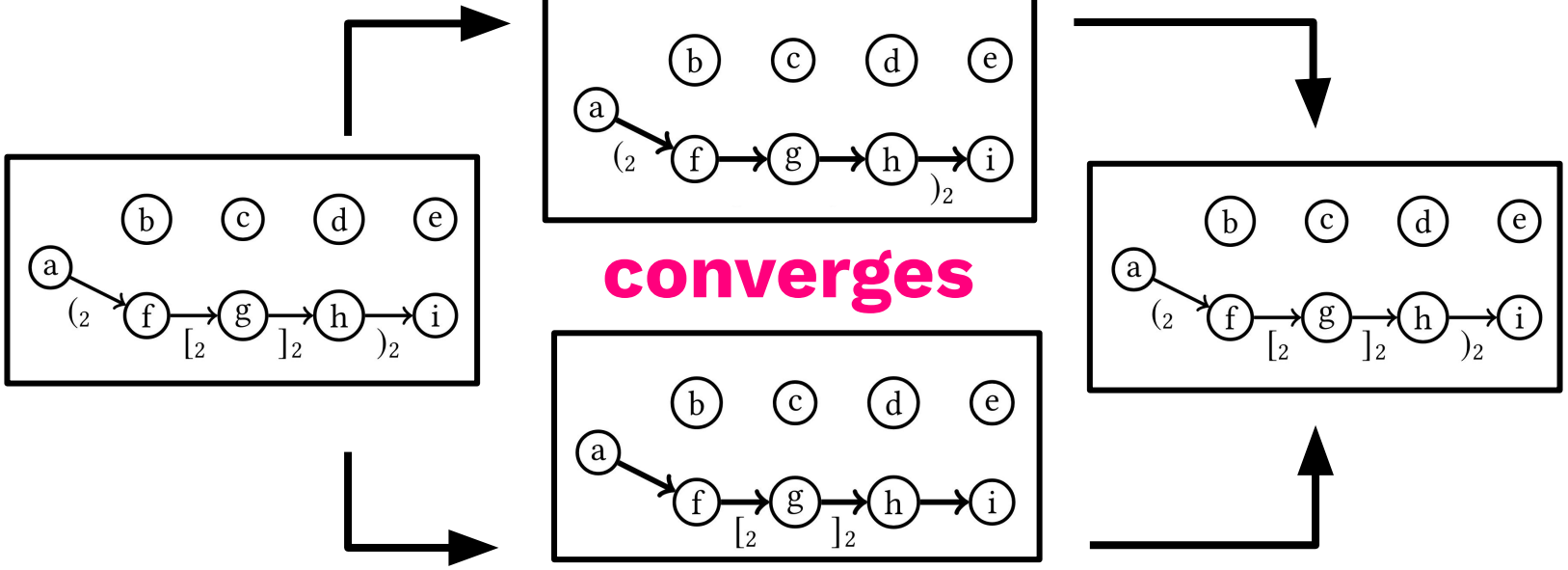
Mutual refinement



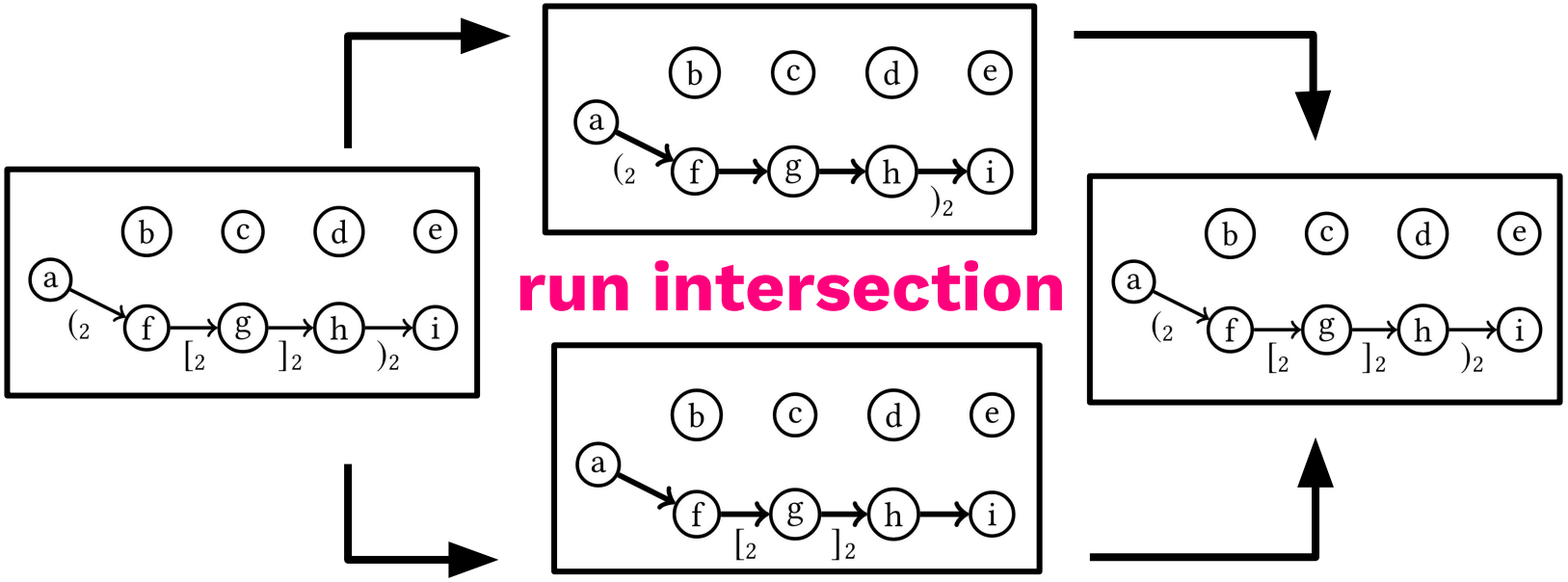
Mutual refinement



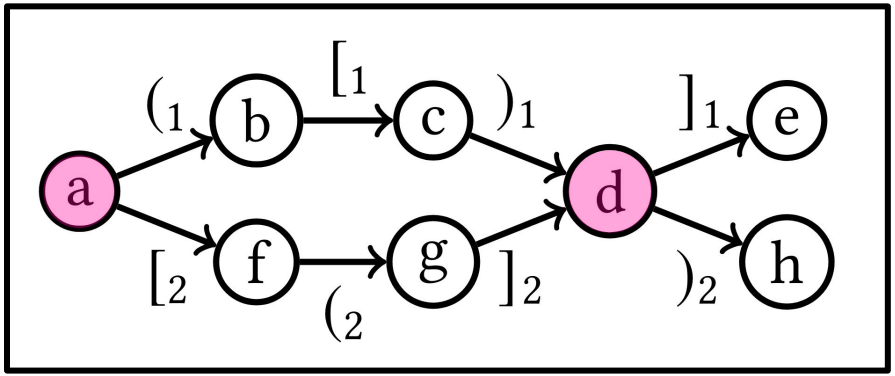
Mutual refinement



Mutual refinement

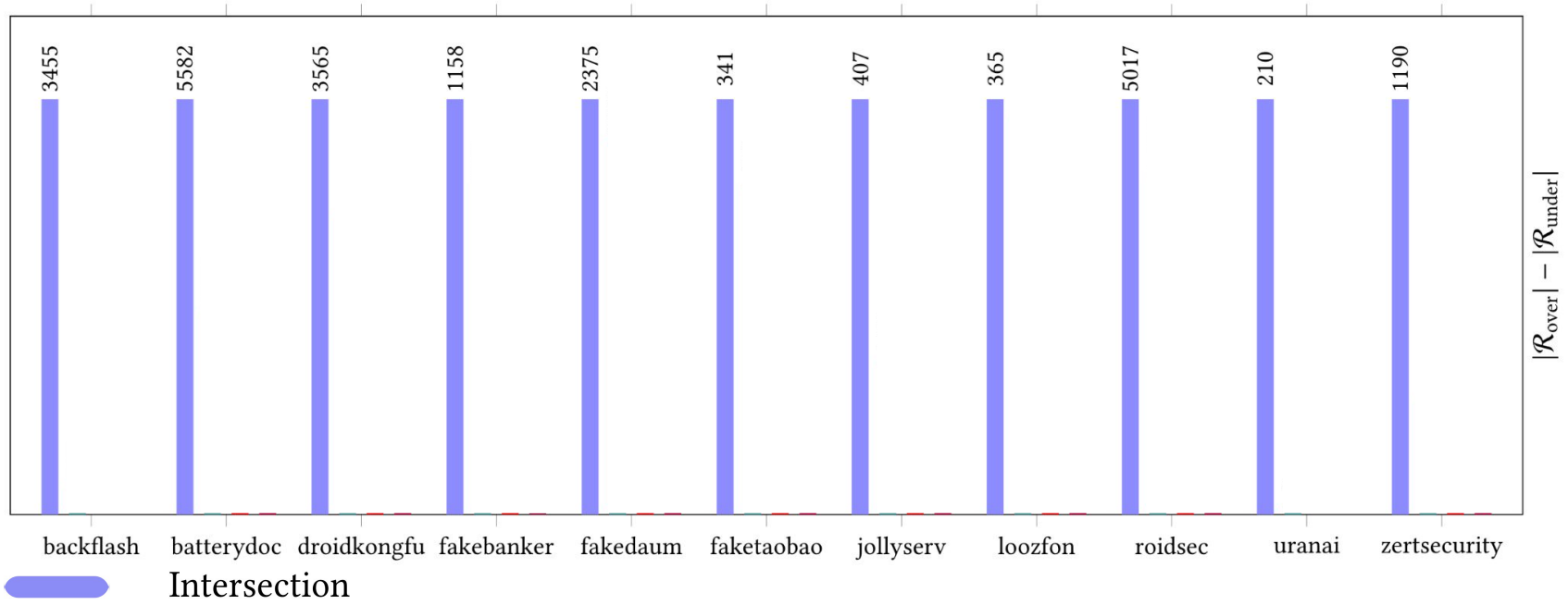


Mutual refinement

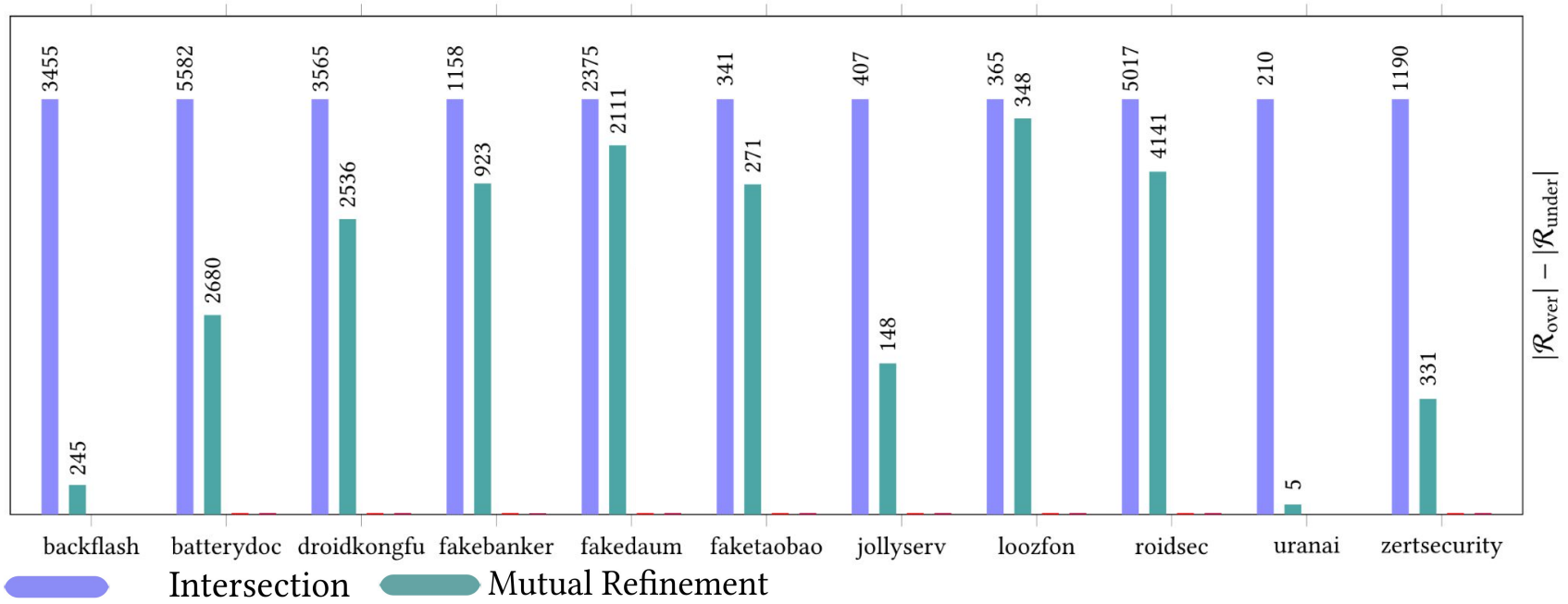


Path from **a** to **d** is still misidentified

And how does it perform?



Potential false positives per benchmark (normalized by worst method)



Our improvements

Augmenting the Context-Free Language

We have been considering “only” parentheses and “only” brackets

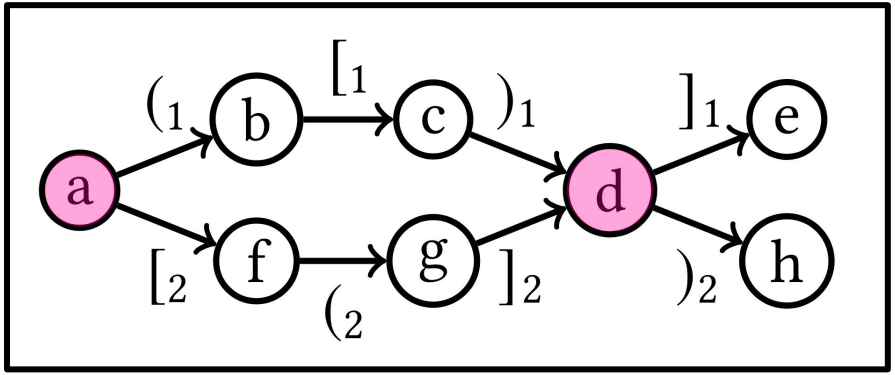
Can we add bracket information to the parenthesis CFL while still keeping it a CFL?

$$S(\epsilon)$$

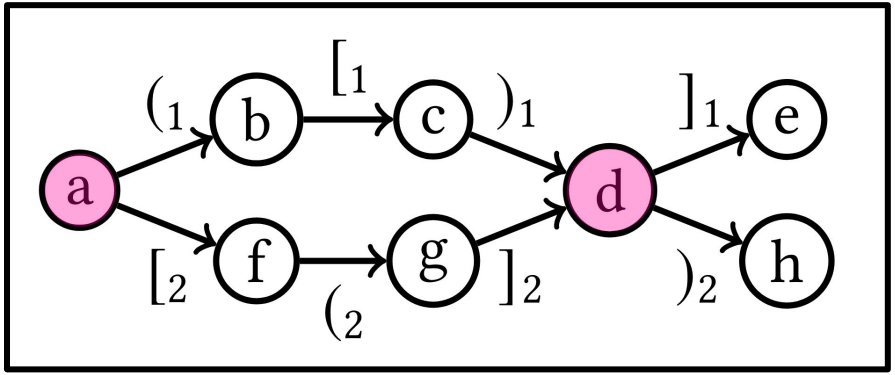
$$S((_i x)_i) \leftarrow S(x)$$

$$S(x_1 x_2) \leftarrow S(x_1), S(x_2)$$

Why does a not reach d?



Why does a not reach d?



All paths from **a** to **d** are of odd length

Augmented CFL-v1

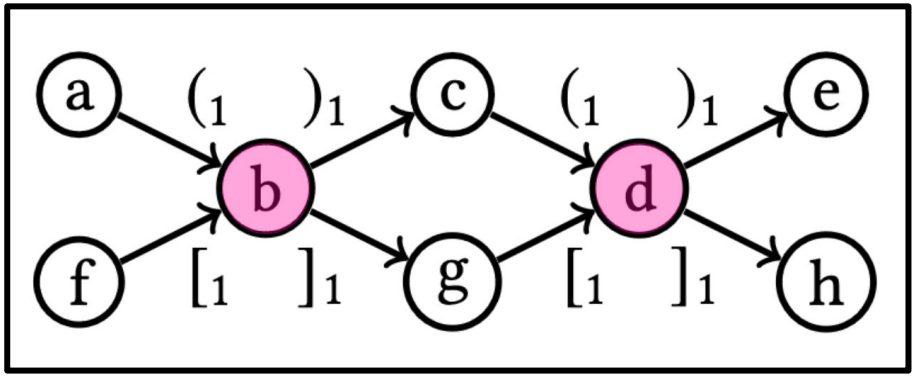
Identify strings that:

- The parenthesis are well-nested
- The number of bracket characters is even

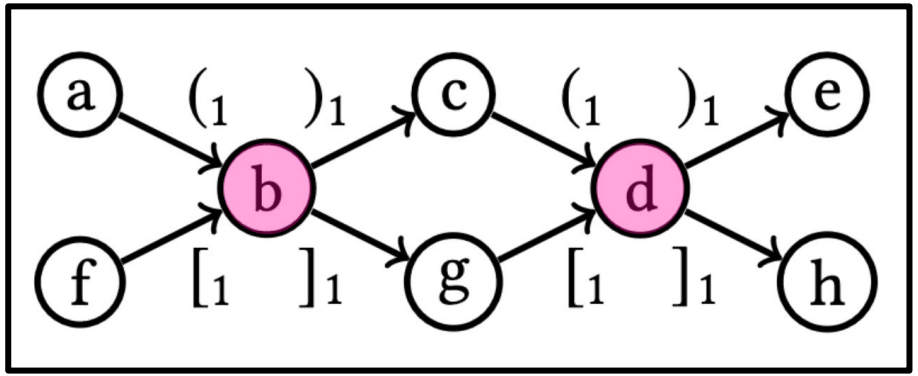
Run mutual refinement for this grammar and the bracket equivalent:

- The brackets are well-nested
- The number of parenthesis characters is even

Why does b not reach d?



Why does b not reach d?



Every path from **b** to **d** starts with a **)** or **]**

Augmented CFL-v2

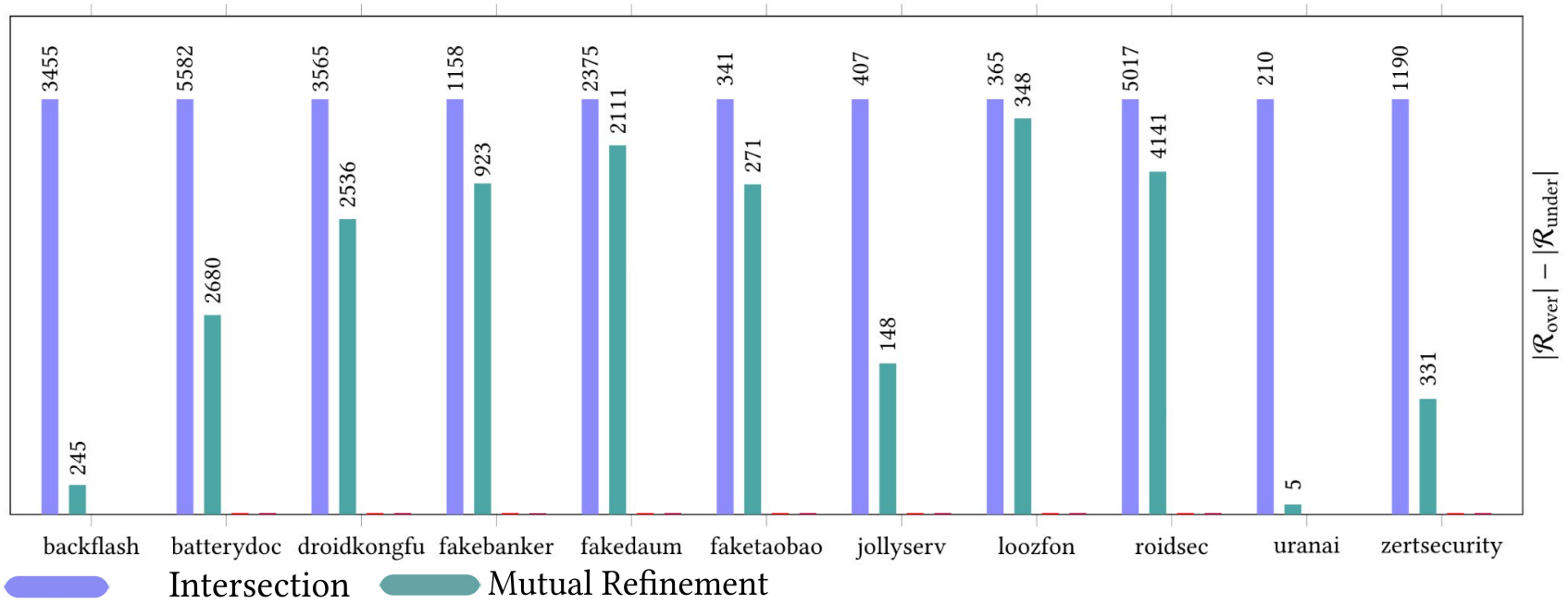
Identify strings that:

- The parenthesis are well-nested
- The number of bracket characters is even
- The first bracket character is **[** and the last is **]**

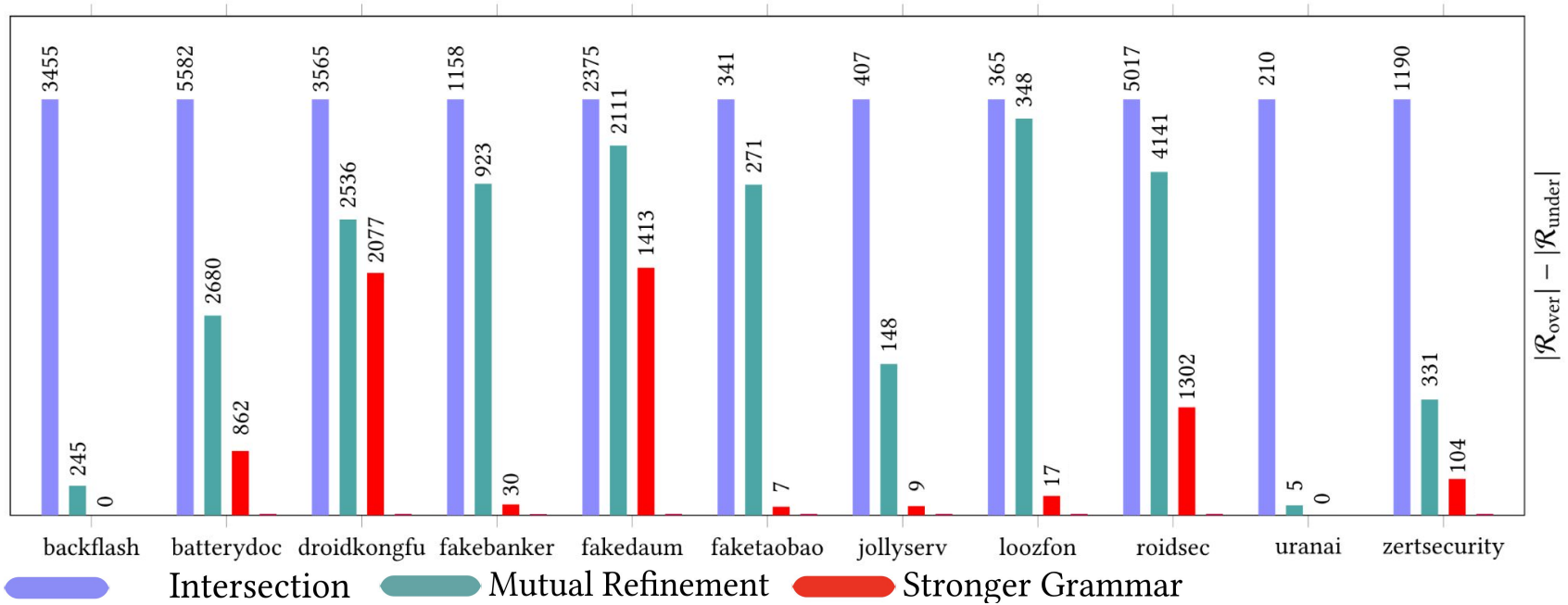
Run mutual refinement for this grammar and the bracket equivalent:

- The brackets are well-nested
- The number of parenthesis characters is even
- The first parenthesis character is **(** and the last is **)**

And how does it perform?

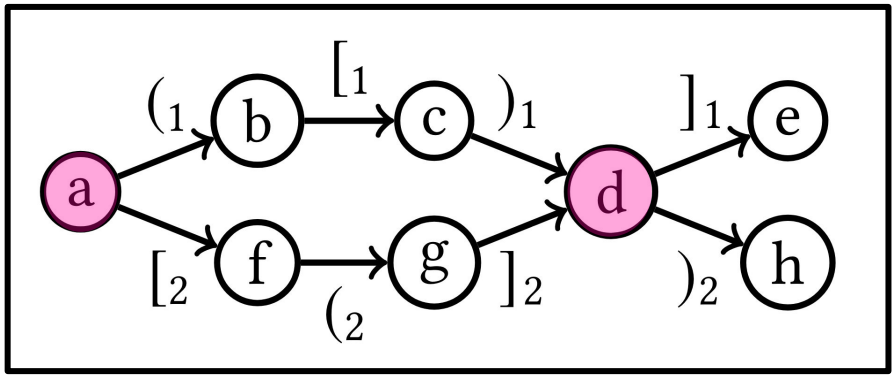


Potential false positives per benchmark (normalized by worst method)



On-Demand setting

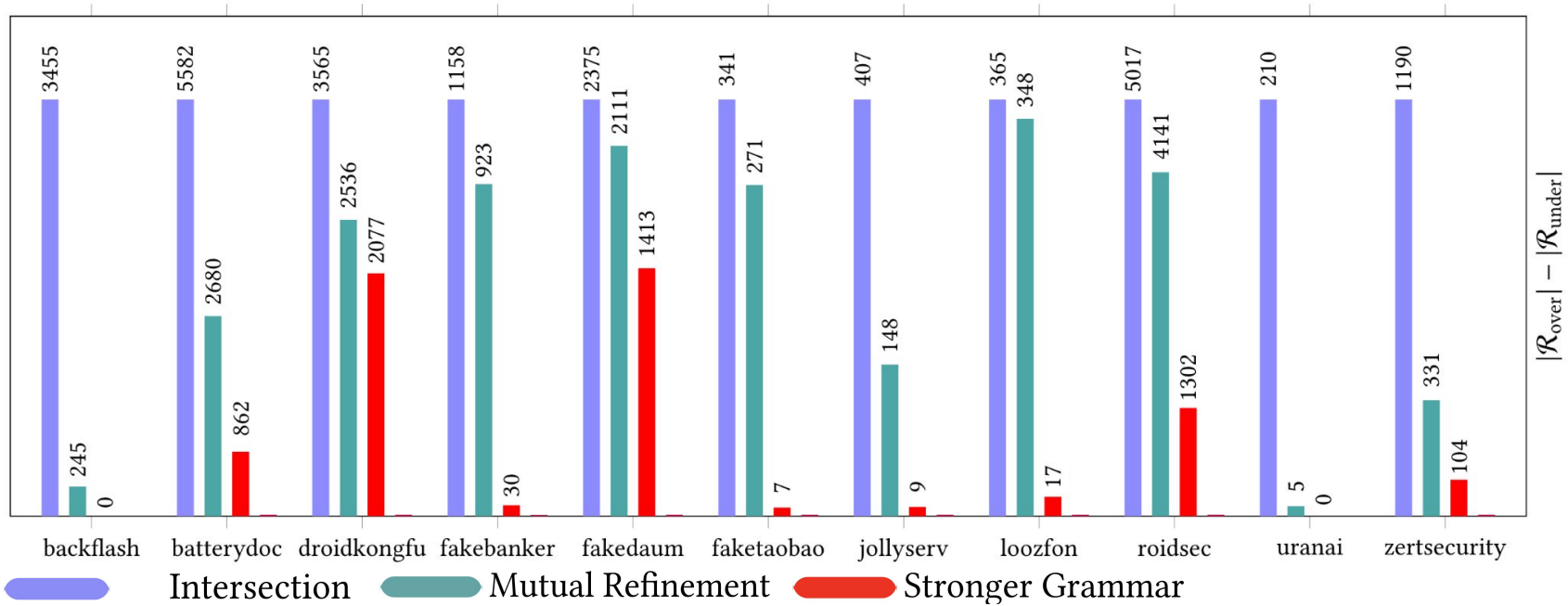
Why do we not remove the “bad” edges from a to d?



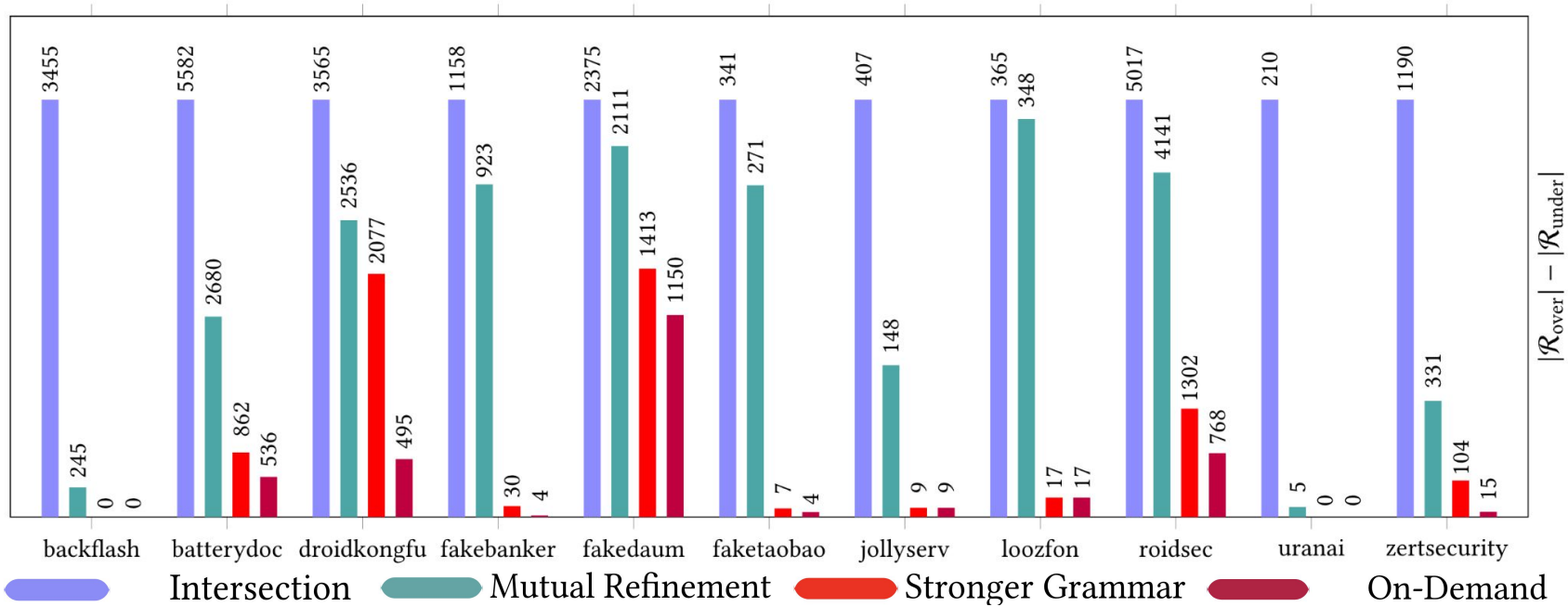
Only check edges from **a** to **d** → run “on-demand”

- Query for a single potential reachable pairs
- Does variable **a** reach variable **b**?
- Mutual refinement excludes edges not a valid path from **a** to **b**, using the refined grammar

And how does it perform?



Potential false positives per benchmark (normalized by worst method)



Conclusion and additional considerations

Runtime considerations

- Our grammar-augmenting methods are significantly more precise than the previous best alternative, while only being slower by a constant factor

- On-demand setting increases precision and it is faster for on-demand queries than the all-pairs version

Overall results

We designed strictly stronger methods for overapproximating Interleaved Dyck Reachability

On average, the number of potential false positives goes from **55%** to **12%** compared to the best previous alternative